A Centralized AI-Powered Thesis Library Using Retrieval-Augmented Generation

---

A Thesis

Presented to the Faculty of the

College of Computing Studies, Information and Communication Technology

Isabela State University

Echague, Isabela

In Partial Fulfillment of the

Academic Requirements for the Degree of

Bachelor of Science in Computer Science (BSCS)

Data Mining Track

By:

Ahron John F. Barlis

Carlo Rossi P. Gallardo

Chapter 1

Introduction

1.1 Background of the Study

At Isabela State University, specifically within the College of Computing Studies, Information and Communication Technology (CCSICT), students rely on manual navigation of digital soft copies and physical hardbound materials in the college library. Current traditional methods involve rigid keyword-based systems that depend solely on the abstract section of a study, failing to capture the full semantic context and technical depth of the research.

This reliance on manual browsing and abstract only searching leads to inefficient literature reviews and a high risk of unintentional topic duplication. While students may attempt to use general Large Language Models (LLMs) to improve synthesis, these public models lack access to CCSICT's private repository. Without access to localized data, standard LLMs are prone to hallucinations, generating false citations instead of providing verifiable local studies. Research in intelligent archiving (Custer & Sethi, 2017) highlights that static repositories often become "data graveyards" when they lack interactive retrieval mechanisms.

From a data mining perspective, extracting value from unstructured thesis manuscripts requires advanced text-mining methodologies. Classical data mining strictly relies on structured, tabular datasets, which fail to capture the semantic depth of academic research. However, this study utilizes embedding models to perform Unstructured Text Mining. By converting raw academic text into high dimensional mathematical vectors, the system computationally mines semantic relationships and contextual proximity utilizing mathematical distance metrics like Cosine Similarity, which traditional keyword mining algorithms cannot achieve.

To address this gap, the study implements a Retrieval-Augmented Generation (RAG) architecture using the LangChain framework. The system utilizes PyMuPDF to extract text directly from digitized CCSICT theses, indexing the full text in a Supabase vector database. LangChain's orchestration then feeds relevant contextual data into a Large Language Model via the Gemini API. This closed-loop RAG implementation ensures that the AI generates responses grounded in verified institutional data (Lewis et al., 2020). By enabling semantic search across entire documents rather than relying solely on abstracts, the architecture also addresses the "Lost in the Middle" phenomenon (Liu et al., 2024), where critical information is often buried within lengthy documents.

The proposed Web-Based Intelligent Thesis Library functions as an indirect knowledge retrieval system, enhancing the efficiency of RRL writing and enabling instant topic validation by providing AI-synthesized, citation-backed responses rather than requiring students to manually browse individual thesis documents. Researchers and faculty will benefit from automated redundancy checks, ensuring that new studies build upon existing work rather than duplicating it. The system's processing speed will be monitored using LangSmith to trace latency, and its response accuracy will be evaluated using the Ragas framework (Es et al., 2024) to verify that the AI demonstrably minimizes hallucinations, thereby maintaining the integrity of all retrieved institutional data.

1.2 Objectives of the Study

General Objective

To develop and implement a Centralized AI-Powered Thesis Library using a Retrieval-Augmented Generation (RAG) AI architecture to solve the semantic gap and improve research retrieval performance in terms of time and management for the College of Computing Studies, Information and Communication Technology (CCSICT).

Specific Objectives

Specifically, the study aims to:

1. Develop a knowledge retrieval model that integrates Retrieval-Augmented Generation (RAG) and a Large Language Model (LLM) using the LangChain framework.

2. Compare the performance of the baseline model, or Standard LLM, versus the proposed model (RAG + LLM) strictly in terms of factual accuracy and hallucination mitigation using real institutional queries.

3. Apply the developed model (LLM + RAG) for the development of the Centralized AI-Powered Thesis Library System.

4. Evaluate the system's internal quality using the ISO/IEC 25010 software product quality standard through automated software testing tools based on the following criteria:

    4.1 Functional Suitability

    4.2 Performance Efficiency

    4.3 Reliability

    4.4 Maintainability

1.3 Scope and Delimitation

The primary focus of this study is the engineering and comparative evaluation of a knowledge retrieval model using Retrieval-Augmented Generation (RAG) for the College of Computing Studies, Information and Communication Technology (CCSICT) at Isabela State University, Echague Campus.

The technical and experimental scope of this research includes the following parameters:

- *Data Digitization**:*** The study will involve the conversion of physical hardbound copies and unstructured digital softcopies from the CCSICT archives into searchable PDF formats suitable for semantic indexing. Scanning physical manuscripts to PDF is an operational prerequisite carried out before ingestion: the system's digitization phase extracts and, where necessary, OCRs text from an uploaded PDF for semantic indexing, and does not itself perform scanning or emit a new searchable PDF.

- *System Architecture:* The system will utilize the LangChain framework as the primary orchestration layer. Gemini will provide the embeddings model to process the documents, and Supabase will serve as the vector database for document chunking and semantic indexing. The Gemini API will provide the Large Language Model (LLM) for text synthesis.

- *Experimental Testing:* Execution of a quantitative comparative test to measure and evaluate retrieval performance specifically calculated in terms of factual accuracy and hallucination mitigation between a standard baseline LLM and the proposed RAG + LLM architecture.

- *System Integration and Evaluation:* Integration of the finalized RAG model into a web-based Thesis Library System, followed by a purely technical evaluation using the ISO/IEC 25010 software product quality standard focusing strictly on internal quality characteristics: Functional Suitability, Performance Efficiency, Reliability, and Maintainability, executed through automated testing tools.

To maintain technical feasibility, ensure controlled experimental variables, and uphold academic integrity, the researchers established the following delimitations:

- *Corpus Restrictions:* The vector database is strictly limited to theses generated within the CCSICT department, comprising both student (undergraduate) and faculty manuscripts, each labelled by category and filterable in browsing, chat retrieval, and analytics. The evaluation corpus for Objective 2 remains exactly the fifty (50) undergraduate theses selected under the corpus protocol; faculty-category manuscripts are excluded from that locked corpus by definition. Research outputs from other colleges, external institutions, or other ISU campuses are entirely excluded from the dataset.

- *External Knowledge Isolation:* To actively prevent AI hallucinations and guarantee epistemic fidelity, the retrieval system is architecturally restricted from searching the open internet. It operates exclusively as a closed-domain retrieval assistant utilizing only local institutional data.

- *Content Generation Limits:* The system is explicitly programmed to retrieve and synthesize existing archived data. It will not generate original research content, automatically write thesis chapters, or construct independent academic arguments.

- *Network Dependency:* The proposed architecture requires a continuous, active internet connection to communicate with the cloud-based vector database and the LLM API. Consequently, offline functionality or local-network-only deployment is completely outside the parameters of this project.

- *Dataset Volume Limit:* The dataset will be limited to fifty (50) thesis documents obtained from the CCSICT library, focusing only on selected undergraduate studies. This sample size was purposely determined based on the volume of accessible and complete undergraduate manuscripts currently available in the CCSICT archives, ensuring proportional representation across the department's academic tracks. The study will not include the mass digitization or inclusion of the entire historical thesis archive; however, the system architecture is designed to scale incrementally as additional documents are digitized in future iterations.

- *Complex Data Extraction Limits:* Complex visual formats such as raw Entity-Relationship Diagrams (ERDs), unformatted code blocks, and unstructured image-based tables that fail layout-aware PDF parsing will not be semantically indexed to prevent vector space corruption. The system will primarily rely on the surrounding explanatory text for these visuals.

- *Indirect Access Model:* The system is designed as an indirect thesis library. Users will not have the capability to directly view, download, or browse the full-text content of any thesis document stored in the database. The system exclusively provides AI-generated responses that synthesize and cite relevant information from the archived theses. This design preserves the intellectual property of the original thesis authors while still enabling knowledge discovery.

- *Duplication Parameter Limit:* To prevent topic redundancy, the system evaluates new submissions against CCSICT archived data using an 85% cosine similarity threshold. If a proposed research query meets or exceeds this limit, the system flags it as a potential duplicate to encourage novelty. At this threshold, the integrated language model automatically alerts the user that the topic already exists and displays the exact

similarity percentage. To provide immediate context for both the student and their faculty adviser, the model also generates a brief summary of the matched archival study. The system reports two distinct figures: the highest passage similarity found against the archive, and the matched-chunk coverage, being the percentage of the new manuscript's chunks whose nearest archived neighbour met the 85% threshold. Coverage drives an advisory verdict of clear, review_suggested below 50%, or high_overlap at 50% and above. The verdict is advisory only: the system never automatically accepts or rejects a proposed topic, and the decision remains with the student and their faculty adviser.

1.4 Significance of the Study

The findings and outputs of this study will provide distinct advantages to the academic community of ISU Echague, particularly in modernizing local knowledge governance. The beneficiaries include:

Undergraduate Students. The proposed system will serve as an indirect retrieval assistant, reducing the time required to manually search through physical archives and unstructured digital files. Through semantic search capabilities, students will receive AI-synthesized responses that summarize relevant methodologies, related literature, and prior project scopes with traceable citations to the original thesis sources, without requiring direct access to complete thesis manuscripts. This improves the quality of literature reviews, minimizes unintentional topic duplication, and ensures access to reliable, locally sourced citations.

Faculty Members and Research Advisers. For instructors, the system provides a data-driven tool to validate the novelty of proposed student topics. Advisers can query the database to cross-reference new proposals against years of accumulated CCSICT theses, streamlining the title defense and approval process.

The CCSICT Department. The project establishes a structured, digitized knowledge base that preserves and protects the intellectual property of the college. By centralizing thesis data into a vector database accessible only through AI-mediated, indirect retrieval, the department transitions away from deteriorating physical copies and fragmented digital storage while ensuring that full thesis content is not directly exposed or downloadable, ensuring long-term institutional memory and compliance with modern data banking standards.

Future Researchers. Academically, this study contributes empirical data to the field of information retrieval. By executing a comparative factual accuracy test between a baseline LLM and a RAG-augmented LLM, future computer science researchers can utilize the results of this experiment as a technical baseline for scaling localized AI search engines in other academic settings.

1.5 Definition of Terms

To ensure clarity and precision, the following terms are defined based on how they are used within the context of this study. These terms are categorically organized in logical progression, prioritizing the core system architecture down to the evaluation metrics:

Centralized AI-Powered Thesis Library System. This refers to the final software application developed in this study. It is a web-based, AI-powered research assistant deployed for the CCSICT department that provides indirect access to CCSICT student and faculty thesis knowledge through semantically grounded, citation-backed AI responses rather than direct document browsing or downloading.

Retrieval-Augmented Generation (RAG). The core computational architecture proposed in this research algorithmically restricts the AI synthesis engine to generate answers based

exclusively on retrieved vector embeddings from the localized CCSICT thesis archives, thereby ensuring locally cited outputs.

LangChain. The primary open-source orchestration framework utilized in the system backend to programmatically link the user's search query, the Supabase vector database, and the Gemini LLM into a unified data retrieval pipeline.

Unstructured Text Mining. The algorithmic process of extracting high-value semantic data and hidden patterns from unformatted text documents (PDFs) through natural language processing and vectorization.

Document Chunking. The programmatic process of dividing digitized CCSICT thesis manuscripts into smaller, discrete text blocks utilizing LangChain to enable precise semantic indexing within the vector database.

Vector Database. The cloud infrastructure, specifically Supabase with the pgvector extension, engineered to store the high-dimensional mathematical representations (embeddings) of the digitized CCSICT thesis corpus, allowing for rapid similarity searches.

Semantic Gap. The limitation observed in traditional library catalog systems, such as OPAC, that rely strictly on literal keyword matching, which frequently fails to interpret the conceptual and contextual intent behind a student’s research query.

Semantic Search. The data retrieval technique that locates institutional information based on the mathematical proximity of their contextual meaning within a vector space, rather than relying on exact word matches.

Cosine Similarity. The specific mathematical metric used by the Supabase pgvector database to calculate the geometric distance and semantic relevance between the user's research query vector and the embedded document vectors.

Baseline Model. The standalone Large Language Model, specifically the standard Gemini API utilized as the control variable. It is evaluated without access to the local database to scientifically establish an empirical baseline of hallucination severity to benchmark factual accuracy against the proposed RAG architecture.

AI Hallucination. In this study, this refers to the generative error where an unaugmented Large Language Model fabricates academic citations, methodologies, or data when queried about ISU Echague's institutional research, producing convincing but mathematically unverified information.

ISO/IEC 25010. The international software quality framework used by the researchers to quantitatively assess the final web system, specifically measuring its internal quality characteristics Functional Suitability, Performance Efficiency, Reliability, and Maintainability through automated software testing and static code analysis rather than subjective user evaluation.

Duplicate Parameter. The predefined mathematical threshold set at 85% cosine similarity. It serves as the algorithmic trigger that allows the system to identify and flag a student's research query as highly redundant compared to an existing CCSICT study, ensuring new research builds upon rather than copies existing work.

Chapter 2

Theoretical Framework

2.1 Review of Related Literature

2.1.1 Retrieval-Augmented Generation (RAG) Architecture

To bridge the semantic gap of traditional systems, the computational linguistics community has converged on the Retrieval-Augmented Generation (RAG) architecture. RAG structurally separates an AI's linguistic reasoning capabilities from its knowledge storage, relying on external retrieval rather than internal memory. Lewis et al. (2020) introduced the original RAG framework, which connects a language model to a searchable index of

documents. Their tests across several tasks showed that RAG produces more accurate answers than a language model working from memory alone, ensuring that generated responses are strictly grounded in verified databases.

Khaliq et al. (2024) studied the optimization of academic queries using retrieval-augmented large language models. They explicitly used the LangChain framework, recursive character text splitters, and vector databases, finding that text-splitting chunk sizes of 700 to 800 tokens optimize LLM responses for academic datasets. Mezhlumyan developed a RAG system for academic papers using a two-layer vector search. The project detailed how to bypass token limits when querying long academic PDFs by utilizing an HNSW index and SBERT to first find relevant thesis abstracts and then narrow down to specific paragraphs (Mezhlumyan, 2024).

Arivazhagan et al. (2023) argued that dense retrievers can generalize poorly out-of-domain and proposed a hybrid hierarchical retrieval approach. Their approach combined sparse and dense retrievers across stages, improving robustness for specialized vocabulary and varying user queries. Supabase documented the use of pgvector for embeddings and vector similarity search. Their technical documentation provided implementation guidance for combining a library repository with a vector search backend, demonstrating the foundational data architecture required for such deployments (Supabase, 2026). These studies show that a robust RAG architecture requires careful orchestration of document loaders, semantic chunking, and multi-layered vector searches to process academic literature efficiently.

Algorithmic Flow of Retrieval-Augmented Generation (RAG)

The RAG architecture operates through a bipartite algorithmic process: the Retrieval Phase and the Generation Phase.

In the Retrieval Phase, a user's natural language query is first processed by an embedding model, specifically Gemini Embedding (models/gemini-embedding-001). Unlike traditional keyword tagging, this process falls under the domain of **Unstructured Text Mining**. The model performs deep feature extraction, mapping the semantic meaning of the text into a high-dimensional continuous vector space, typically consisting of 768 dimensions.

Once vectorized, the algorithm executes a nearest-neighbor search against the Supabase pgvector database, which contains the pre-embedded chunks of institutional thesis documents. The retrieval is governed by **Cosine Similarity**, a mathematical distance metric that calculates the cosine of the angle between the query vector (A) and the document vector (B). Mathematically expressed as

$$Cosine\ Similarity = \frac{A \cdot B}{||\ A\ ||||\ B\ ||}$$

Where:

A = Query vector (the user's research question converted into a high-dimensional embedding)

B = Document vector (a pre-embedded thesis chunk stored in the Supabase pgvector database)

$A \cdot B$ = Dot product of vectors A and B

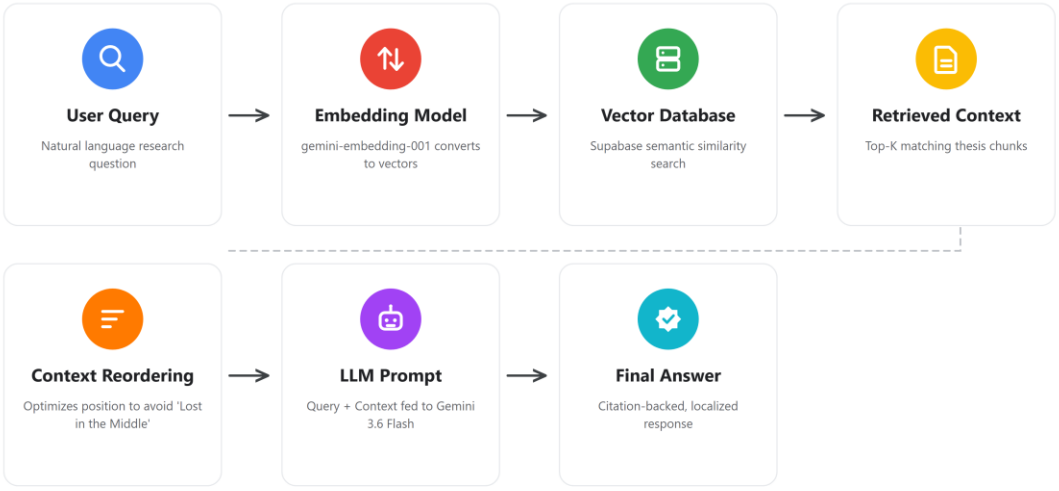
$||\ A\ ||||\ B\ ||$ = Product of the magnitudes of vectors A and B

This formula ensures that documents sharing the closest contextual meaning even if they do not share exact keywords are pulled from the database.

In the Generation Phase, these retrieved top-*k* document chunks are concatenated with the user's original query to form an augmented prompt. Instead of relying on its parametric memory, which risks hallucination, the LLM algorithm utilizes its self-attention mechanisms to synthesize an answer strictly derived from the appended context window. This algorithmic pipeline mathematically grounds the generated text in the localized data, ensuring verifiability (see Figure 1).

Figure 1

Algorithmic Flow of Retrieval-Augmented Generation (RAG)



2.1.2 Large Language Models (LLMs) and Hallucination Mitigation

While general LLMs possess powerful natural language understanding, their direct application in academic research introduces profound risks of hallucinations, making targeted context retrieval and faithfulness critical. Kovanen (2025) explored the taxonomy of AI errors by distinguishing between factuality contradictions and faithful hallucinations. The study defined faithfulness as the AI's ability to stick strictly to the uploaded context without fabricating outside information. Barnett et al. (2024) identified seven critical failure points when engineering RAG systems, including incomplete retrieval, missed context, and wrong-format outputs, demonstrating the strict engineering constraints required to balance factual grounding and answer coverage. Their findings provide a practical engineering checklist for diagnosing and preventing retrieval failures in production RAG deployments.

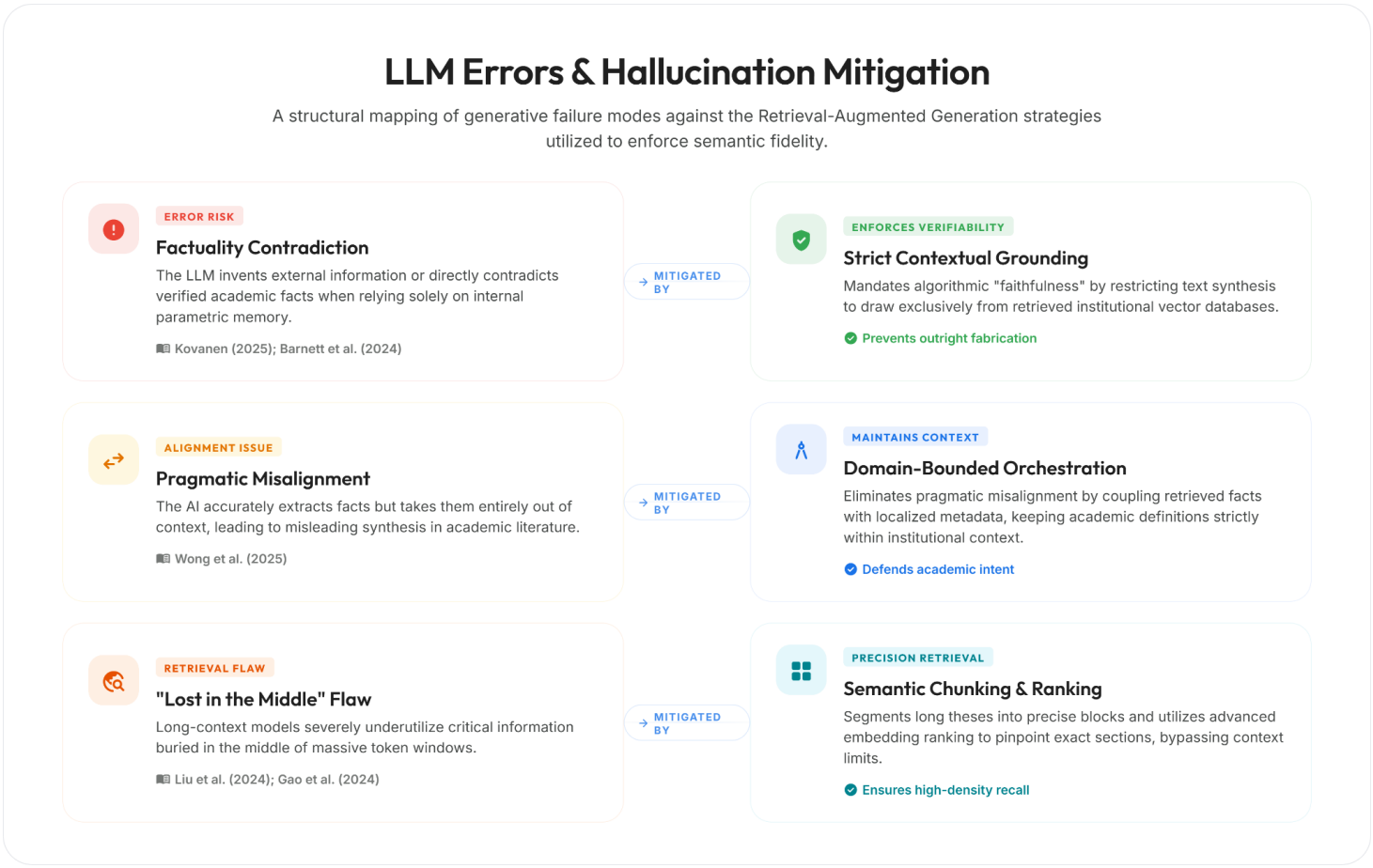
Gao et al. (2024) surveyed a wide range of RAG architectures and noted that knowledge representation strategies particularly how text is semantically chunked significantly influence LLM accuracy and hallucination rates. Their survey synthesized findings from multiple studies demonstrating that semantically chunking texts such as separating methodology from results

yields higher AI accuracy and drastically reduces hallucinations. Wong et al. (2025) warned that RAG systems can suffer from pragmatic misalignment. Their critical paper explained that systems might pull technically accurate facts but take them out of context, leading to misleading answers and becoming dangerous communicators if not properly bound.

Liu et al. (2024) showed that long-context language models underutilize information located in the middle of long prompts. Their study on the "Lost in the Middle" phenomenon strongly supported the need for careful chunking, retrieval ordering, and semantic search to pinpoint relevant sections rather than stuffing chapters into prompts. Gao et al. (2024) similarly addressed this phenomenon, noting how information buried in hundreds of pages requires targeted retrieval to maintain model effectiveness. These studies show that while RAG mitigates basic hallucinations, ensuring true faithfulness requires advanced semantic chunking, context reordering, and an awareness of pragmatic misalignment to guarantee accurate academic synthesis (see Figure 2).

Figure 2

Identified LLM Error Behaviors and Corresponding RAG Mitigation Strategies



2.1.3 Traditional Library Systems and the Semantic Gap

Traditional library systems, such as Online Public Access Catalogues (OPAC), are foundational to academic institutions but often struggle with the semantic complexity of modern research. These systems typically operate on rigid Boolean logic and literal keyword matching, which limits their ability to process natural language queries effectively. Bevara et al. (2025) explored the shift from traditional keyword-based Boolean library searches to Natural Language Retrieval-Augmented Generation (RAG) systems. Their work discussed metadata indexing and real-time query processing, proving that traditional systems like OPAC fail at handling complex student queries. The authors found that this failure creates a strict demand for semantic AI search capabilities.

Xuan and Shaw (2025) presented a real-world case study of an academic library implementing RAG to allow users to search their institutional repository using natural language instead of rigid metadata. Their study showed that top-tier institutions globally are replacing traditional OPAC systems with RAG, indicating that this modernization is a necessary technological transition for academic archives. These studies show that traditional library systems operating strictly on literal word matching cannot interpret the semantic intent behind complex academic searches, leading to high rates of unintentional topic duplication and the necessity of semantic AI modernization.

Local Context

Within the local context of the Philippines, the limitations of traditional library systems are heavily documented. A study by Doliente et al. (2023) conducted at a higher educational institution in Mindanao revealed that the majority of students rarely utilize the traditional Online Public Access Catalog (OPAC) due to user dissatisfaction, rigid search functionalities, and an inability to interpret complex research queries. Additionally, Armilla and Abangan (2025) assessed the digitization of academic libraries in the Philippines, highlighting that while State Universities are transitioning from physical hardbound copies to digital PDFs, they face massive technological bottlenecks in information retrieval.

To resolve this, Philippine academic sectors are beginning to explore AI. Sambrano et al. (2025) studied state university students in Region I and found a massive shift in library patronage, noting that students now prefer AI tools over traditional OPACs strictly because of "faster information retrieval" and natural language understanding. Even at the national level, the Supreme Court of the Philippines Library recently initiated the redevelopment of their E-Library to transition away from keyword-matching, integrating AI natural language processing to allow users to search legal precedents using conversational queries (Supreme Court of the

Philippines, 2024). These local studies validate the urgent necessity for State Universities like ISU Echague to modernize their unstructured thesis archives using semantic AI architectures.

2.1.4 Data Preprocessing and Information Overload

The inability of traditional systems to facilitate deep, interactive querying directly leads to the manifestation of the "data graveyard" phenomenon, where unstructured academic data becomes buried and unusable. Custer and Sethi (2017) highlighted that static repositories often become "data graveyards" because they lack interactive retrieval mechanisms. Their research pointed out that vast amounts of collected institutional data are stored but rarely accessed or interpreted due to severe usability hurdles.

Vercruysse et al. (2024) focused on extracting semantic graphs and structuring Linked Data into plain objects for developers. Their study highlighted the complexity of raw data extraction and supported the argument that converting unstructured academic data, such as digitized PDFs, into structured, readable formats requires specialized semantic lenses. Gokdemir et al. (2025) detailed a High-Performance Computing (HPC) RAG workflow that parsed over 3.6 million scientific PDFs. They introduced a layout-aware PDF parser called Oreo and a query-aware encoder, explaining the massive computational challenges of extracting text from hardbound scanned theses and defending the need for dedicated data preprocessing stages.

Aytar et al. (2025) focused on using GROBID for data cleaning, semantic chunking, and an abstract-first retrieval method. Their framework helped navigate literature without information overload by quickly scanning thesis abstracts before performing a deep dive into the full PDF content, saving significant computational power. These studies show that unstructured academic texts pose a significant challenge, and that without proper layout-aware

parsing and abstract-first retrieval strategies, documents risk becoming completely inaccessible data graveyards.

2.1.5 Educational RAG Systems and Traceable Citations (Related Studies)

The application of RAG within educational domains provides institutional support and enforces academic verifiability through explicitly traceable citations. Li et al. (2025) reviewed dozens of studies applying RAG across educational scenarios through a systematic survey. The survey identified recurring challenges such as hallucination and evaluation gaps, strengthening the argument that RAG is an established and highly necessary approach for educational support.

Oreški and Vlahek (2024) described the development of an AI chatbot for student support based on RAG. Their case study reported prototyping choices motivated by domain knowledge gaps, illustrating how RAG can be operationalized as an academic assistant strictly grounded in institutional content. Besrour et al. (2025) detailed a system providing fine-grained in-line citations for scientific question-answering. Their multi-agent RAG system allowed users to trace claims back to specific sentences, validating that RAG can eliminate hallucinations by providing explicit, verifiable citations from local archives.

Gao et al. (2023) introduced the ALCE benchmark for generating text with citations. They argued that citations improve verifiability and constrain hallucinations, demonstrating that responses must treat citations as a first-class output to support academic claims effectively. These studies show that educational RAG applications significantly benefit students by acting as domain-bounded assistants, and that enforcing explicit in-line citations is critical for maintaining academic integrity and verifiability.

2.1.6 System Evaluation, Privacy, and Security

The successful deployment of a Web-Based Intelligent Thesis Library necessitates rigorous, quantifiable evaluation methodologies and strict security protocols to protect institutional intellectual property. Santra et al. (2025) discussed building a locally deployed conversational search tool called the BiblioGPT prototype. Their work emphasized protecting user privacy and institutional data by avoiding the risks of public cloud LLMs, proving that limiting AI to a local database through "Knowledge Isolation" is a secure practice.

Bodea et al. (2026) created a taxonomy of privacy risks in RAG systems. Their study detailed data leakage, prompt leakage, and database leakage, explaining how processing institutional data locally mitigates these vulnerabilities. The Open Web Application Security Project (OWASP) listed major security risks for LLM applications, including prompt injection and sensitive data disclosure. Their standards provide authoritative justification for strict access controls and data separation in RAG pipelines (OWASP, 2025).

To further mitigate institutional data exposure, the system will utilize the Gemini API under deployment terms that prohibit the retention or use of submitted data for model training, ensuring compliance with Google's API data governance policy (Google, 2024). This guarantee is bounded by the provider actually serving each call: the architecture permits generation to be routed through an OpenAI-compatible gateway without changing the model, and a third-party operator is not covered by those terms. The direct Google route will therefore be used for the formal evaluation and for any processing of the governed corpus, and the release fingerprint will record which route produced each result. Embedding calls are never routed and reach Google in every configuration. Brown et al. (2025) published a highly comprehensive review of 128 RAG studies that categorized evaluation metrics. Their systematic literature review justified the use of precision, recall, F1-score, and LLM-as-a-judge methodologies for evaluating RAG architectures objectively. Es et al. (2024) introduced the RAGAs automated

evaluation framework. Their system demo emphasized evaluating both retrieval and generation modules using reference-free metrics without requiring massive human annotation. Ru et al. (2024) proposed RAGChecker, a fine-grained framework for diagnosing RAG. Their work highlighted claim-level evaluation, helping to separate errors caused by the retriever missing evidence from errors caused by the model hallucinating despite having the correct chunk. These studies show that deploying a thesis library requires strict local knowledge isolation to prevent data leakage, paired with modular automated frameworks to continuously evaluate retrieval precision and generation faithfulness.

2.1.7 Synthesis of the Literature

The reviewed literature converges on several critical findings that collectively establish the theoretical and technical foundation for this study. Across the domains of RAG architecture, LLM hallucination mitigation, traditional library systems, data preprocessing, educational applications, and evaluation frameworks, a consistent pattern emerges: existing academic retrieval systems are fundamentally limited by the absence of an integrated, semantically intelligent pipeline.

First, the studies on RAG architecture (Lewis et al., 2020; Khaliq et al., 2024; Mezhlumyan, 2024; Arivazhagan et al., 2023) collectively establish that connecting language models to external document indexes through vector-based retrieval produces significantly more accurate and verifiable outputs than relying on parametric memory alone. These findings are directly reinforced by the hallucination mitigation literature (Kovanen, 2025; Barnett et al., 2024; Wong et al., 2025), which identifies the specific failure modes including fabricated citations, pragmatic misalignment, and the "Lost in the Middle" phenomenon (Liu et al., 2024; Gao et al., 2024) that occur when language models operate without grounded retrieval. Together, these two bodies of research demonstrate that RAG is not merely an enhancement

but a necessary architectural constraint for any AI system intended to handle academic queries with factual integrity.

Second, the literature on traditional library systems (Bevara et al., 2025; Xuan & Shaw, 2025; Doliente et al., 2023; Sambrano et al., 2025) consistently documents the failure of keyword-based OPAC systems to interpret the semantic intent behind complex research queries. This limitation directly connects to the data preprocessing literature (Custer & Sethi, 2017; Gokdemir et al., 2025; Aytar et al., 2025), which explains that without layout-aware parsing and semantic structuring, institutional archives become inaccessible "data graveyards." The convergence between these findings establishes a cause-and-effect relationship: rigid keyword systems create the access barrier, while unprocessed data worsens it.

Third, the educational RAG literature (Li et al., 2025; Oreški & Vlahek, 2024; Besrour et al., 2025; Gao et al., 2023) demonstrates that RAG successfully serves as a domain-bounded academic assistant when paired with traceable citations. However, the evaluation and security literature (Brown et al., 2025; Es et al., 2024; Ru et al., 2024; Santra et al., 2025; Bodea et al., 2026; OWASP, 2025) cautions that deployment must include strict local knowledge isolation and automated evaluation frameworks to prevent data leakage and ensure continuous generation faithfulness.

Despite these well-documented findings, the reviewed studies address these challenges largely in isolation. RAG studies focus on general question-answering benchmarks rather than localized institutional theses. Hallucination studies examine model-level errors but rarely contextualize them within academic library retrieval. Traditional library studies identify the semantic gap but stop short of implementing AI solutions. Educational RAG studies operate on curated datasets rather than raw, unstructured, digitized thesis manuscripts. No single study integrates all these components: document digitization, semantic chunking, vector-based

retrieval, LLM synthesis with traceable citations, hallucination mitigation, and automated evaluation into a unified, closed-loop pipeline specifically tailored for an institutional thesis archive.

2.1.8 Gap and Relevance to the Study

This is the specific gap that the present study addresses. Existing university library systems rarely integrate a complete, closed-loop RAG pipeline specifically designed to process digitized academic theses using vector databases for semantic indexing and LLMs for citation-backed synthesis. Most commercial LLMs suffer from ungrounded hallucinations because they lack access to local institutional databases, while traditional OPACs lack the natural language understanding needed for complex academic discovery. The proposed system bridges this gap by utilizing PyMuPDF to extract text directly from digitized CCSICT theses, a Supabase vector database for semantic indexing, and an LLM constrained entirely to this localized context. By combining these technologies, the system will provide faster RRL writing, easy topic validation, automated redundancy checking, and will statistically minimize AI hallucinations regarding local references, thereby cultivating a high-standard research culture.

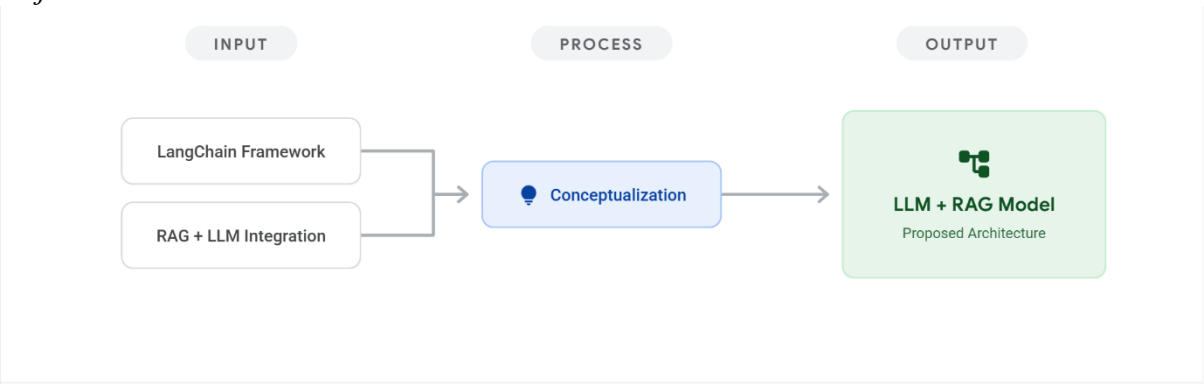
From a Data Mining perspective, the vectorization of unstructured thesis PDFs constitutes a form of unsupervised feature extraction, a core data mining technique. The subsequent nearest-neighbor retrieval using Cosine Similarity mirrors the classification and clustering paradigms foundational to data mining methodology. This positions the study firmly within the Data Mining track by applying mathematical text-mining operations to extract actionable knowledge from previously inaccessible institutional archives.

2.2 Conceptual Framework

The conceptual framework of this study is structured into four distinct Input-Process-Output (IPO) models, directly mapping to the study's specific objectives. This modular approach outlines the logical progression from theoretical conceptualization to final system evaluation.

Figure 3

Objective 1: Foundational Architecture



Input: The *LangChain Framework* and *RAG + LLM Integration* theoretical guidelines serve as the primary foundational software dependencies required to conceptualize the new knowledge retrieval model.

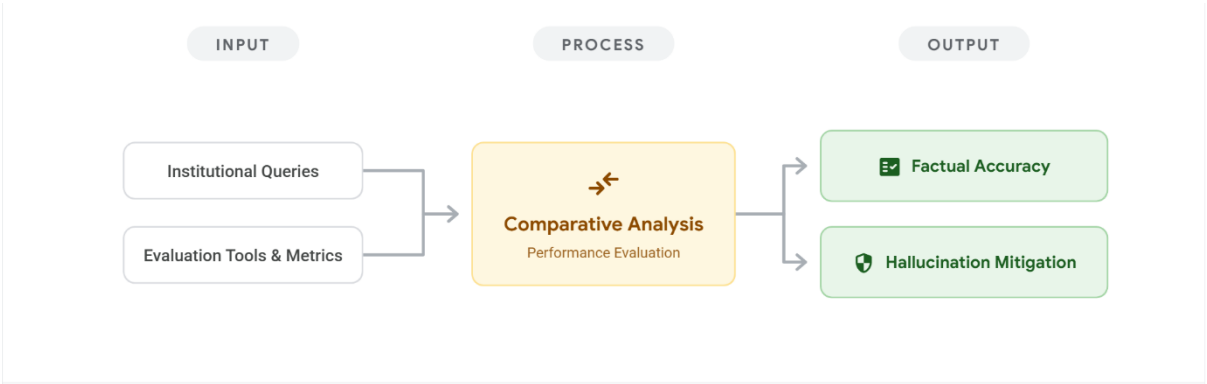
Process: The *Theoretical Conceptualization* phase is conducted by integrating Retrieval-Augmented Generation capabilities over base Large Language Model architectures to construct the retrieval pipeline.

Output: The direct result of this phase yields the mathematically and structurally defined *LLM + RAG Model (Proposed Architecture)*, which acts as the blueprint for subsequent testing.

As illustrated in Figure 3, the Input relies on the *LangChain Framework* and *RAG + LLM Integration* theoretical guidelines, which serve as the primary foundational software dependencies required to conceptualize the new knowledge retrieval model. The Process involves *Theoretical Conceptualization*, conducted by integrating Retrieval-Augmented Generation capabilities over base Large Language Model architectures to construct the retrieval pipeline. The Output of this phase yields the mathematically and structurally defined *LLM + RAG Model (Proposed Architecture)*, which acts as the blueprint for subsequent testing.

Figure 4

Objective 2: Comparative Performance Analysis



Input: *Institutional Queries* represent the primary real-world data utilized as functional inputs, which are assessed using established *Evaluation Tools & Metrics* to create a structured comparative testing environment.

Process: A rigorous *Comparative Analysis* follows a logical progression of scientific testing, evaluating the side-by-side performance differences between the baseline and the newly integrated model.

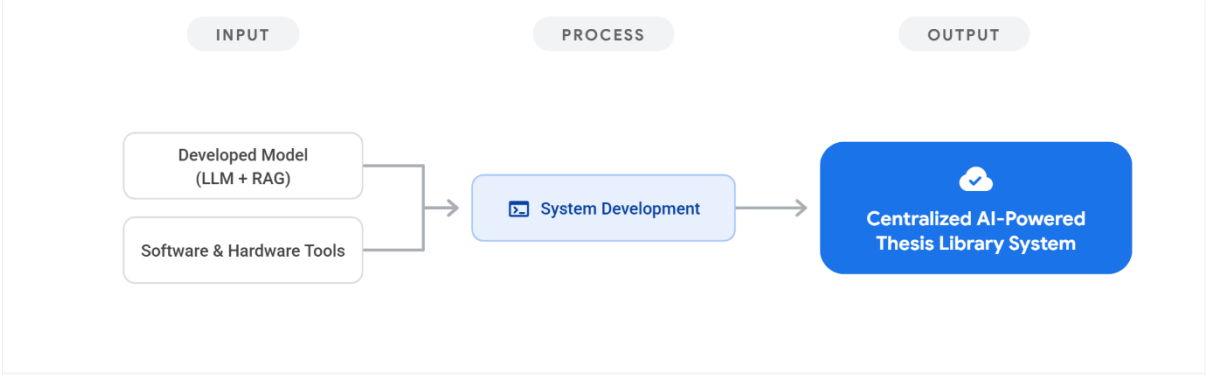
Output: The outputs generated from this phase structure the empirical data into definitive metrics, specifically quantifying the models' capabilities in extending *Factual Accuracy* and enabling *Hallucination Mitigation*.

As illustrated in Figure 4, the Input utilizes *Institutional Queries* representing the primary real-world data, which are assessed using established *Evaluation Tools & Metrics* to create a structured comparative testing environment. The Process involves a rigorous *Comparative Analysis* following a logical progression of scientific testing, evaluating the side-by-side performance differences between the baseline and the newly integrated model. The Output generated from this phase structures the empirical data into definitive metrics,

specifically quantifying the models' capabilities in extending *Factual Accuracy* and enabling *Hallucination Mitigation*.

Figure 5

Objective 3: System Integration



Input: The validated *Developed Model (LLM + RAG)*, combined with requisite *Software & Hardware Tools*, provide the technical fundamentals needed to construct the complete software application.

Process: *System Development* acts as the core full-stack engineering stage. This involves embedding the optimized artificial intelligence framework into a tangible, containerized platform.

Output: The process yields the finalized, operational *Centralized AI-Powered Thesis Library System* as the primary tangible product of the study.

As illustrated in Figure 5, the Input takes the validated Developed Model (LLM + RAG), combined with requisite Software & Hardware Tools, to provide the technical fundamentals needed to construct the complete software application. The Process of System Development acts as the core full-stack engineering stage. This involves embedding the optimized artificial intelligence framework into a tangible, integrated web-based platform. The

Output yields the finalized, operational Centralized AI-Powered Thesis Library System as the primary tangible product of the study.

Figure 6

Objective 4: Formal Quality Evaluation

Input: The completely *Developed System* acts as the final subject requirement, deployed into a controlled testing environment and subjected to automated software testing tools for internal evaluation.

Process: Structured assessment procedures are conducted following the standardized guidelines specified by the *ISO/IEC 25010 software product quality standard.*

Output: The system definitively generates empirical computational metrics, strictly evaluating the deployed software's *Functional Suitability*, *Performance Efficiency*, *Reliability*, and *Maintainability*.

As illustrated in Figure 6, the Input for the final phase is the completely *Developed System*, which acts as the final subject requirement deployed into a controlled testing environment and subjected to automated software testing tools. The Process involves structured computational assessment procedures conducted following the standardized guidelines specified by the *ISO/IEC 25010* software product quality standard. Finally, the Output definitively generates empirical computational metrics, strictly evaluating the deployed software's *Functional*

Suitability, *Performance Efficiency*, *Reliability*, and *Maintainability* through automated software testing tools.

Chapter 3

Methodology

3.1 Materials

3.1.1 Software

For the programming and deployment phases, the researchers will utilize a specific stack of development tools and cloud services. Visual Studio Code will serve as the primary integrated development environment (IDE) used to write, edit, and organize the system's source code.

Table 1

Frontend Development Stack

Technology	Version	Purpose
React	v19.2.8	Will serve as the primary library for developing a component-based, highly responsive user interface for student researchers.

JavaScript (JSX)	ES6+	Will be utilized as the core scripting language for implementing front-end logic and dynamic rendering.
Vite	v8.1.5	Will function as the build tool and development server to ensure rapid hot-module replacement and optimized production bundling
Node.js	v24.18.0	Will provide the JavaScript runtime for the build toolchain, the development server, and the automated test runner.
Tailwind CSS	v4.3.3	Will supply the utility-first styling layer applied consistently across all interface views.
React Router	v8.3.0	Will handle client-side routing between the landing, archive, chat, and administrative views.
TanStack Query	v5.101.4	Will manage server state, caching, and background refetching for API-backed views.
Axios	v1.18.1	Will serve as the HTTP client for all requests from the frontend to the FastAPI backend.

Framer Motion	v12.42.2	Will provide the animation and transition primitives used by the interface components.

Table 2

Backend Software Specifications

Technology	Version	Purpose
FastAPI	v0.139.2	Will be employed as the web framework to build a high-performance REST API capable of handling asynchronous requests.
Pydantic	v2.13.4	Will be utilized for strict data validation and settings management through Python type hinting.
python-multipart	v0.0.32	Will be integrated to facilitate the parsing of multipart form data, specifically for handling document uploads.
python-dotenv	v1.2.2	Will manage environment variables to maintain secure configuration of API keys and database credentials.
Python	v3.14.7	Will serve as the backend runtime, pinned to an exact patch level and asserted at container build time.

pydantic-settings	v2.14.2	Will load and validate server configuration from environment variables at startup.
Uvicorn	v0.51.0	Will run the ASGI server hosting the FastAPI application.
SlowAPI	v0.1.10	Will enforce per-route and default rate limits that protect the API from abuse and runaway cost.
Redis	v8.0.1	Will provide the shared store backing rate limiting and short-lived operational state.
cryptography	v50.0.0	Will supply the cryptographic primitives used for token and secret handling.
PyJWT	v2.13.0	Will encode and verify the JSON Web Tokens used for authenticated sessions.
HTTPX	v0.28.1	Will perform outbound HTTP calls to Supabase and the Gemini API.

Table 3

AI and RAG Orchestration Tools

Technology	Version	Purpose
Supabase	v2.31.0	Will be utilized as the client interface for connecting the back-end to the PostgreSQL-based cloud database.
pgvector	v0.7.0 (Managed via Supabase)	Will be enabled within the database to support vector similarity searches for semantic document retrieval.
langchain-google-genai	v4.3.1	Will facilitate the secure integration of Google's Generative AI models into the system's backend.
Gemini 3.6 Flash	gemini-3.6-flash	Will be employed as the primary Large Language Model (LLM) for synthesizing intelligent, context-aware responses.

Gemini Embedding	models/gemini-embedding-001 (768 dimensions)	Will be utilized to generate high-dimensional vector embeddings for indexing and searching document contents.
Gemini 3.5 Flash Lite	gemini-3.5-flash-lite	Will handle bounded verdict and metadata-extraction work that does not require the primary generation model.
langchain-core	v1.5.1	Will provide the Runnable and prompt abstractions that compose the retrieval-to-generation chain.
langchain-text-splitters	v1.1.2	Will supply the RecursiveCharacterTextSplitter used to chunk extracted manuscript text.
langchain-openai	v1.4.1	Will provide the OpenAI-compatible chat client used only when generation is routed through a gateway; embeddings are never routed through it.

tiktoken	v0.13.0	Will provide the cl100k_base tokenizer used as a fixed proxy for measuring chunk sizes.
PyMuPDF	v1.28.0	Will extract text and layout information directly from uploaded PDF manuscripts.
tesserocr	v2.10.0	Will bind the Tesseract OCR engine for image-based scanned pages.
Pillow	v12.3.0	Will handle page image preparation on the OCR path.
LangSmith	v0.10.10	Will trace and log pipeline latency and token consumption during evaluation runs.

Table 4

Software Testing and Quality Assurance Tools

Technology	Version	Purpose
PyTest	v9.1.1	Will be employed as the automated testing framework for executing unit tests and integration tests to verify the functional correctness of API endpoints and RAG pipeline operations.
Apache JMeter	v5.6.3	Will be utilized as a load-testing tool to simulate concurrent user requests and measure API response times, throughput, and system performance under stress conditions.
SonarQube	Community Build 26.7.0.124771	Will be deployed as a continuous static code analysis platform to detect code smells, logic flaws, unhandled exceptions, and potential reliability vulnerabilities in the backend codebase.
Pylint	v4.0.6	Selected to generate automated, objective code quality scores by evaluating the structural maintainability, modularity, and strict typing compliance of the Python backend.

ESLint	v9.39.5	Selected to enforce strict ECMAScript standards and evaluate the structural maintainability and technical debt mitigation of the React frontend codebase.
Node.js test runner	v24.18.0 (node --test)	Will execute the frontend unit test suite using the runtime's built-in test runner.
Playwright	v1.61.1	Will drive end-to-end browser tests across the critical user journeys of the interface.
axe-core for Playwright	v4.12.1	Will perform automated accessibility checks within the end-to-end suite.
SonarScanner CLI	v8.0.1.6346	Will submit the analysis to the SonarQube server that produces the retained Reliability evidence.

3.1.2 Hardware

To develop and test the system effectively, the researchers will use a laptop with specific hardware parts. Table 5 specifies the development workstation rather than a deployment requirement: all embedding and generation work is performed remotely through

the Gemini API and OCR is CPU-bound, so the discrete GPU is not used by the architecture. These components are chosen to make sure the software runs smoothly and maintains a stable connection to the cloud-based AI services. The table below lists the primary hardware used in this study.

Table 5

Hardware Specifications

Hardware Component	Description & Specifications	Gap
CPU	Intel Core i5 13th Gen	Feasible
GPU	Nvidia RTX 4050	Feasible
RAM	16GB 4800MHz	Feasible
Storage	1TB M.2 Gen 4	Feasible
Network Adapter	Wi-Fi 6 Wireless capability	Feasible
Network High-speed Broadband Internet	Required for continuous real-time communication with the Supabase vector database and the Gemini API.	

3.1.3 Data

The data for this study will come directly from the local archives of the College of Computing Studies, Information and Communication Technology (CCSICT). The data will include past student (undergraduate) and faculty thesis projects from the department, each labelled by category, using both physical hardbound copies and digital files. The evaluation corpus for Objective 2 remains exactly the fifty (50) student manuscripts. Converting physical

copies into PDF is an operational prerequisite carried out before ingestion; the system extracts and, where necessary, OCRs text from an uploaded PDF and does not itself perform scanning.

Before adding the data to the system, all files will be converted into searchable PDF formats. The original PDF files will be kept in a Supabase storage bucket. Each thesis usually has around 15,000 to 20,000 words, making the PDF file size about 300 KB to 500 KB.

To enable the AI to search through this information quickly, the system will convert the text into vector data. Since vector embeddings represent high-dimensional numerical formats optimized for semantic similarity, they are substantially more storage-efficient than the original unstructured PDFs. After this conversion, the estimated size for the vector data of each thesis will be significantly lower, at approximately 100 KB to 200 KB. This vector data will be securely stored and managed using Supabase as the primary database. This specific vector dataset will act as the local knowledge base. This will ensure that the AI will only use real institutional research during the testing phase instead of external internet sources.

3.2 Methods

3.2.1 Research Design

This study will employ a computational systems design and quantitative comparative framework to evaluate the efficiency and accuracy of the retrieval architecture. The objective is to measure the performance variance between two distinct computational pathways when processing complex academic queries.

The study will utilize purposive sampling in selecting fifty (50) undergraduate thesis documents from the CCSICT archives. The selected dataset will include both physical manuscript copies and digital soft copies, ensuring representation across the CCSICT academic catalog: the Data Mining, Web and Mobile Application Development, and Network and

Security specializations, together with the BSDSA, BSIS and BLIS programs, which carry no specialization and are therefore identified by program code. This approach ensures that the evaluation covers diverse technical vocabularies relevant to the domain. Every archived manuscript carries a student or faculty category label, and the Objective 2 evaluation corpus is restricted to student (undergraduate) manuscripts.

Two testing environments will be established. The control environment will utilize an unaugmented Large Language Model, specifically the baseline Gemini API, relying exclusively on its parametric memory for query resolution. The experimental environment will deploy the proposed Retrieval-Augmented Generation (RAG) system. Within this architecture, the language model will be algorithmically constrained; it will use LangChain processes to retrieve highly similar vector embeddings from the Supabase database before synthesizing a response. The effective model set, comprising the chat, verdict, and embedding models together with the generation and retrieval parameters, will be frozen in a signed release fingerprint for the duration of the evaluation, so that every reported result is attributable to one exact configuration. The fingerprint will additionally record which provider served the generation calls, so that a reported result cannot be attributed to the wrong route.

During execution, identical institutional research queries will be processed through both models simultaneously. The generated outputs will be systematically benchmarked. The primary variable measured will be factual accuracy, specifically evaluating the RAG architecture's computational ability to mitigate hallucinations by generating verifiable citations derived strictly from the local CCSICT vector dataset.

To execute the automated Ragas evaluation objectively, the researchers will manually curate a 'Golden Dataset' comprising thirty to fifty (30–50) standardized academic queries. These queries will be paired with human-verified 'Ground Truth' answers derived directly from

the thesis corpus. Both the baseline LLM and the RAG model will process this exact dataset. To eliminate researcher bias and ensure academic neutrality, these 'Ground Truth' answers will be validated by a panel of three CCSICT faculty members prior to serving as the absolute mathematical benchmark for the Ragas evaluation.

To ensure methodological traceability, Table 6 presents a direct mapping of each specific objective to its corresponding tools, methods, and expected outputs. This mapping serves as the structural guide for the subsequent subsections.

Table 6

Methodological Mapping of Objectives to Tools and Methods

Specific Objective	Tools and Technologies	Method / Procedure	Expected Output
Objective 1	LangChain framework, Gemini Embedding, Supabase pgvector, PyMuPDF, Tesseract OCR	Data Digitization, Semantic Indexing and Metadata Injection, RAG Pipeline Development (Section 3.2.3, Phases 1–3)	Developed LLM + RAG Knowledge Retrieval Model
Objective 2	Ragas framework, LangSmith, Golden Dataset (30–50	Quantitative Comparative Analysis using	Empirical performance data on Answer

	queries), Shapiro-Wilk test	paired-samples t-test or Wilcoxon Signed-Rank Test (Section 3.2.1, 3.2.4, 3.2.5)	Correctness (the paired baseline-versus-RAG comparison metric), reported alongside Faithfulness and Context Precision as RAG-only diagnostics
Objective 3	React v19.2.8, Vite v8.1.5, FastAPI v0.139.2, Pydantic, Supabase, Gemini API	Full-stack System Integration of the RAG backend with the web-based frontend (Section 3.2.3, Phase 4)	Operational Web-Based Thesis Library System
Objective 4	PyTest, Apache JMeter, SonarQube, Pylint, ESLint, ISO/IEC 25010 internal quality criteria	Automated software testing: unit testing (PyTest), load testing (JMeter), static code analysis (SonarQube), and code quality linting (Pylint/ESLint) (Section 3.2.4)	ISO/IEC 25010 internal quality ratings for Functional Suitability, Performance Efficiency, Reliability, and Maintainability

3.2.2 *Software Development Life Cycle*

To systematically guide the engineering of the Centralized AI-Powered Thesis Library System, this study will adopt the Iterative Development Model as its Software Development Life Cycle (SDLC) methodology. The Iterative Model structures the development process into repeated cycles, where each iteration produces a refined, progressively functional version of the system. This approach is selected because the RAG architecture requires empirical tuning of parameters such as chunk sizes, overlap configurations, similarity thresholds, and prompt structures, which necessitate multiple cycles of development, testing, and refinement before final deployment.

As illustrated in Figure 7, the development process is divided into the following iterative phases:

1. *Planning and Requirements Analysis.* The researchers will identify the functional and non-functional requirements of the system based on the study's specific objectives and the operational needs of the CCSICT department. This phase includes defining the system's core features: document upload and digitization, semantic search, AI-powered query response with traceable citations, and administrative management. The software and hardware requirements outlined in Section 3.1 will serve as the technical constraints for this phase.

2. *System Design.* Based on the gathered requirements, the researchers will design the system architecture, database schema, user interface wireframes, and the data flow of the RAG pipeline. The four-layer system architecture illustrated in Figure 8 (User Interface Layer, Application Layer, Embedding Layer, and Data Storage Layer) will serve as the primary design blueprint. This phase will also produce the use case diagrams and entity-relationship diagrams necessary for structuring the Supabase database and defining user interactions.

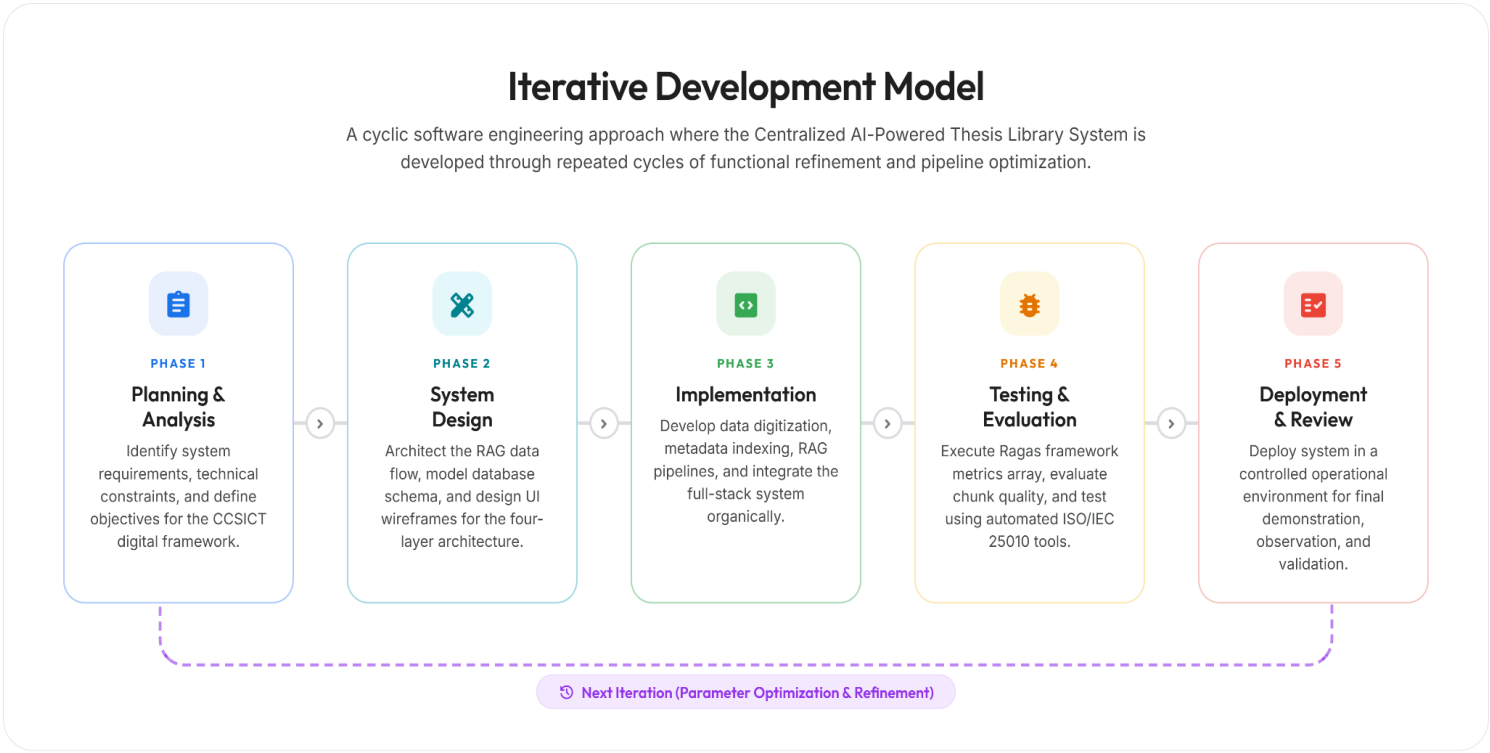
3. *Implementation.* The system will be developed in iterative cycles following the four technical phases described in Section 3.2.3: (a) Data Digitization, (b) Semantic Indexing and Metadata Injection, (c) RAG Pipeline Development, and (d) System Integration. Each phase will be implemented, tested, and validated before proceeding to the next. Within the RAG Pipeline Development phase specifically, multiple iterations will be conducted to optimize the chunk size, embedding quality, retrieval threshold, and context reordering configurations.

4. *Testing and Evaluation.* At the end of each iteration, the system will undergo testing to verify functional correctness and retrieval performance. The comparative analysis between the baseline LLM and the proposed RAG + LLM model (Objective 2) will be executed during this phase using the Ragas framework. Upon completion of the final iteration, the fully integrated system will be subjected to formal quality evaluation against the ISO/IEC 25010 standard using the automated testing tools specified in Section 3.2.4.

5. *Deployment and Review.* The finalized system will be deployed in a controlled environment for demonstration and final validation. Post-deployment observations and evaluation results will be documented as part of the study's findings. Each iteration refines the output of the previous cycle, ensuring that the final system is empirically optimized before formal evaluation. This iterative approach ensures that technical issues identified during testing such as low retrieval precision, suboptimal chunking, or high response latency are corrected in subsequent development cycles rather than discovered only after full system completion.

Figure 7

Iterative Software Development Life Cycle Model



3.2.3 System Procedures

The construction of the system will be divided into four primary computational phases:

1. *Data Digitization:* Physical, hardbound thesis manuscripts from the CCSICT archives will be converted into machine-readable digital formats (PDFs). That conversion is an operational prerequisite performed outside the application; the system's digitization phase begins with an uploaded PDF. The system will primarily use the PyMuPDF library to extract text from digital-native PDF files. For physical manuscripts that result in image-based scanned PDFs, an Optical Character Recognition (OCR) engine (Tesseract OCR) will be integrated. To mitigate the 'Garbage In, Garbage Out' (GIGO) risk, a programmatic data cleaning pipeline will be executed following text extraction.

Layout-aware parsing and Regular Expressions (Regex) will be deployed to sanitize the data by stripping out OCR artifacts, page numbers, headers, footers, and the excluded sections (Table of Contents and Bibliographies). This ensures the embedding model only receives high-quality, noise-free text inputs. To prevent vector space corruption from faded physical copies, the Regex pipeline will programmatically discard any text chunk where non-alphanumeric characters exceed 15% of the string. Additionally, when complex visuals like ERDs are bypassed by the parser, the system will inject a standardized text placeholder “FIGURE REDACTED FOR SEMANTIC INDEXING” to maintain contextual integrity for the generative model.

2. *Semantic Indexing and Metadata Injection:* The cleaned text will be processed by LangChain's RecursiveCharacterTextSplitter, configured to an empirically optimized 800-token chunk size with a 100-token overlap to preserve contextual boundaries. Chunk sizes will be measured with the local cl100k_base tokenizer as a fixed, reproducible proxy, because the Gemini tokenizer is not publicly available; chunk boundaries are therefore exact for that proxy and approximate for the embedding model. Crucially, before these chunks are embedded, Metadata Tagging will be applied. Each text chunk will be programmatically tagged with a JSON object containing its source document's Title, Author, Track, and Year. Finally, the Gemini Embedding model will convert these enriched blocks into vectors, storing them alongside their metadata in the Supabase database. This strict metadata-to-chunk mapping ensures the generative model has the exact variables needed to produce highly traceable, accurate in-line citations. Citation validation will prove marker validity and coverage, meaning that every substantive claim in an answer carries a valid, in-range citation, but it does not establish semantic entailment between a claim and the passage it cites; faculty verification therefore remains part of the process.

3. *RAG Pipeline Development:* A LangChain Expression Language (LCEL) prompt-to-model chain will be engineered to establish a closed-loop data pipeline. When a user inputs a query, the system will retrieve the top-k most relevant semantic vectors from Supabase. To mathematically solve the 'Lost in the Middle' phenomenon, the RAG pipeline will implement a context reordering algorithm re-implementing LangChain's LongContextReorder, before feeding the text into the Gemini API prompt context. This ensures the most critically relevant vectors are placed at the very beginning and very end of the prompt window, where the LLM's attention mechanism is mathematically strongest. Additionally, the pipeline will enforce a minimum cosine similarity threshold of 0.30 over the five nearest chunks (top-k = 5); if no retrieved chunk exceeds this threshold, the system will explicitly inform the user that no relevant thesis was found rather than generating an ungrounded response. To maintain originality within the academic library, the system automatically screens new submissions against existing documents. It calculates the similarity between texts, and if a new entry reaches an 85% match or higher, the system flags it. This triggers an automated warning for the AI model, ensuring that any potential duplicates are clearly identified in the output along with their exact match percentage. Two figures are reported: the highest passage similarity and the matched-chunk coverage, being the percentage of the new manuscript's chunks whose nearest archived neighbour met the threshold. Coverage drives an advisory verdict of clear, review_suggested below 50%, or high_overlap at 50% and above, and the verdict is advisory only — the system never automatically accepts or rejects a topic.
4. *System Integration:* The backend RAG algorithm will be merged with the frontend web interface to create a cohesive platform where students and faculty can execute literature

reviews and conduct novelty scanning, while administrators generate institutional research analytics.

3.2.4 System Evaluation

To facilitate a rigorous comparative analysis, the researchers will utilize industry-standard Machine Learning Operations (MLOps) frameworks. LangSmith will be integrated within the LangChain orchestration layer to trace and log precise computational latency during the retrieval process. Additionally, to quantitatively measure generation accuracy, the study will employ the Ragas (RAG Assessment) framework to mathematically evaluate the architecture's capacity to maximize factual grounding and minimize out-of-domain hallucinations, calculating Answer Correctness against the faculty-validated ground truth as the paired metric on which the two pathways will be compared, together with Faithfulness and Context Precision as RAG-only diagnostics of grounding and retrieval quality. These Ragas metrics are reference-based rather than reference-free, since Answer Correctness consumes the faculty ground truth directly. Because the baseline pathway has no retriever, it returns no retrieved contexts to rank, so Context Precision cannot be computed for it and will be reported for the RAG pathway alone.

The software product quality of the developed platform will be assessed according to established international guidelines, focusing entirely on internal quality characteristics. The four selected criteria Functional Suitability, Performance Efficiency, Reliability, and Maintainability represent the internal quality characteristics of ISO/IEC 25010 that can be objectively measured through automated instrumentation. External quality characteristics such as Usability, which require subjective end-user evaluation, are excluded from this study's scope to maintain a purely technical, bias-free evaluation methodology. Automated testing tools will evaluate the system based on four specific metrics derived from the ISO/IEC 25010 standard:

- **Functional Suitability:** Evaluated internally by verifying the accuracy of system functions and API endpoints through automated unit testing using PyTest and by conducting algorithmic evaluation of the RAG pipeline using the Ragas framework.

- **Performance Efficiency:** Measured at the architectural level by tracking algorithmic latency and token consumption using LangSmith, and by stress testing API endpoints using Apache JMeter to simulate concurrent system loads.

- **Reliability:** Assessed by conducting continuous static code analysis using SonarQube to identify logic flaws, unhandled exceptions, and potential fault points within the backend architecture.

- **Maintainability:** Evaluated by generating automated code quality and structural complexity scores using linting tools such as Pylint for Python and ESLint for React to ensure code modularity, adherence to standards, and reduced technical debt. Pylint scores are reported against the configuration committed at rag-thesis-backend/.pylintrc, which raises the line-length limit to 120 characters, excludes the test suite from analysis, and disables fourteen message families including the docstring and function-size checks. The reported score is therefore comparable only against that profile.

3.2.5 Statistical Treatment of Data

To ensure a rigorous quantitative analysis, the data gathered from the experimental testing and system evaluation will be subjected to the following statistical treatments:

1.Descriptive Statistics for Automated Internal Evaluation

To interpret the quantitative data gathered from the automated software testing tools evaluating the system's Functional Suitability, Performance Efficiency, Reliability, and

Maintainability, descriptive statistical analysis will be utilized.

Quantitative metrics will be extracted directly from automated tool execution logs, including unit test pass rates expressed as percentages obtained from automated functional testing, average response time measured in milliseconds and throughput in requests per second obtained through load testing, code quality scores expressed as percentages derived from static analysis linting tools, and vulnerability counts identified through security scanning. These metrics will be systematically tabulated. The arithmetic mean will be computed for these continuous variables across multiple automated test iterations to establish the operational performance and structural integrity of the system.

The formula for the arithmetic mean is as follows:

$$\bar{x} = \frac{\sum x_i}{n}$$

Where:

$\bar{x}$ = Arithmetic Mean

$\sum x_i$ = Sum of all recorded metric values across test iterations

n = Total number of automated test iterations

2. Inferential Statistics for AI Model Comparison

To compare the factual accuracy and hallucination mitigation between the baseline model (Standard LLM) and the proposed model (RAG + LLM), inferential statistical analysis will be applied to the evaluation scores generated by the Ragas framework, particularly in terms of Answer Correctness, which is the only metric both pathways can be scored on against the faculty-validated ground truth.

Prior to conducting the paired-samples t-test on the Ragas scores, the Shapiro-Wilk test will be utilized to assess the assumption of normality. If the data is normally distributed, the parametric paired-samples t-test will proceed to determine whether there is a statistically significant difference between the performance of the two models when processing identical institutional queries. If the normality assumption is violated, the non-parametric Wilcoxon Signed-Rank Test will be applied as the robust alternative. A significance level of 0.05 will be used as the strict basis for decision-making.

3.2.6 Ethical Considerations

To uphold ethical standards in research involving institutional data, the researchers will adhere to the following ethical safeguards.

First, the researchers will obtain formal written approval from four authorities prior to accessing and digitizing any thesis manuscript from the college archives: the CCSICT Department Chair for academic scope, the University Librarian for corpus custody and rights, an authorized privacy officer for the data-privacy review, and the thesis adviser for the final methodology and corpus. This institutional clearance will serve as authorization for the use of student-authored academic outputs within a closed, non-commercial research system.

Second, the study will comply with Republic Act No. 10173, or the Data Privacy Act of 2012, by applying deterministic pattern-based redaction of high-risk personally identifiable information (PII) at ingestion as a best-effort technical control, paired with mandatory human privacy review, so that no PII beyond the thesis title, author name, academic track, and year of completion which are already publicly available on the thesis manuscript cover is extracted, stored, or exposed by the system. Additionally, the indirect access model protects intellectual property by preventing users from viewing, downloading, or reproducing any full-text thesis content. Where the deployment routes generation through a third-party gateway rather than

calling Google directly, that operator will be treated as a processor of institutional data and disclosed to the privacy review before any governed manuscript is processed through it.

Third, the system is explicitly designed to function as a retrieval assistant, not as a content generator. Its architecture is programmatically constrained to prevent outputs that could facilitate plagiarism. All AI-generated responses will include traceable citations directing users back to the original thesis source, reinforcing proper academic attribution.

3.3 System Architecture

The technical framework of the Centralized AI-Powered Thesis Library System will be designed to securely process institutional data and execute semantic retrievals. As illustrated in Figure 8, the system architecture will be structured into four primary interactive layers: the User Interface Layer, the Application Layer, the Embedding Layer, and the Data Storage Layer. Figure 8 depicts the experimental retrieval pipeline; the delivered platform surrounds that pipeline with production-grade controls which fall outside the experimental scope but are necessary for institutional deployment. Ingestion in particular runs asynchronously rather than synchronously: an upload is validated and staged privately, enqueued as a durable leased job, and processed by a separate worker that downloads, scans for malware, extracts, chunks, embeds, and screens the manuscript before an atomic commit. Authentication with row-level security, multi-factor administrative access, rate limiting, and malware scanning apply as cross-cutting controls across all four layers.

The workflow will begin at the User Interface Layer, which will accommodate distinct user roles. These comprise a Guest Researcher (unauthenticated, scoped to the evaluation department, with no saved history and an optional one-time bot check), authenticated student and faculty accounts, an administrator role, and a superadmin role for cross-department administration and operations. Upload is administrator-default but may be granted to student

and faculty accounts through a server-owned permission matrix. Administrators will utilize the upload interface to input new thesis PDFs into the system. Concurrently, students, faculty, and researchers will interact with the Web Application UI to input natural language research queries and view the AI-generated responses.

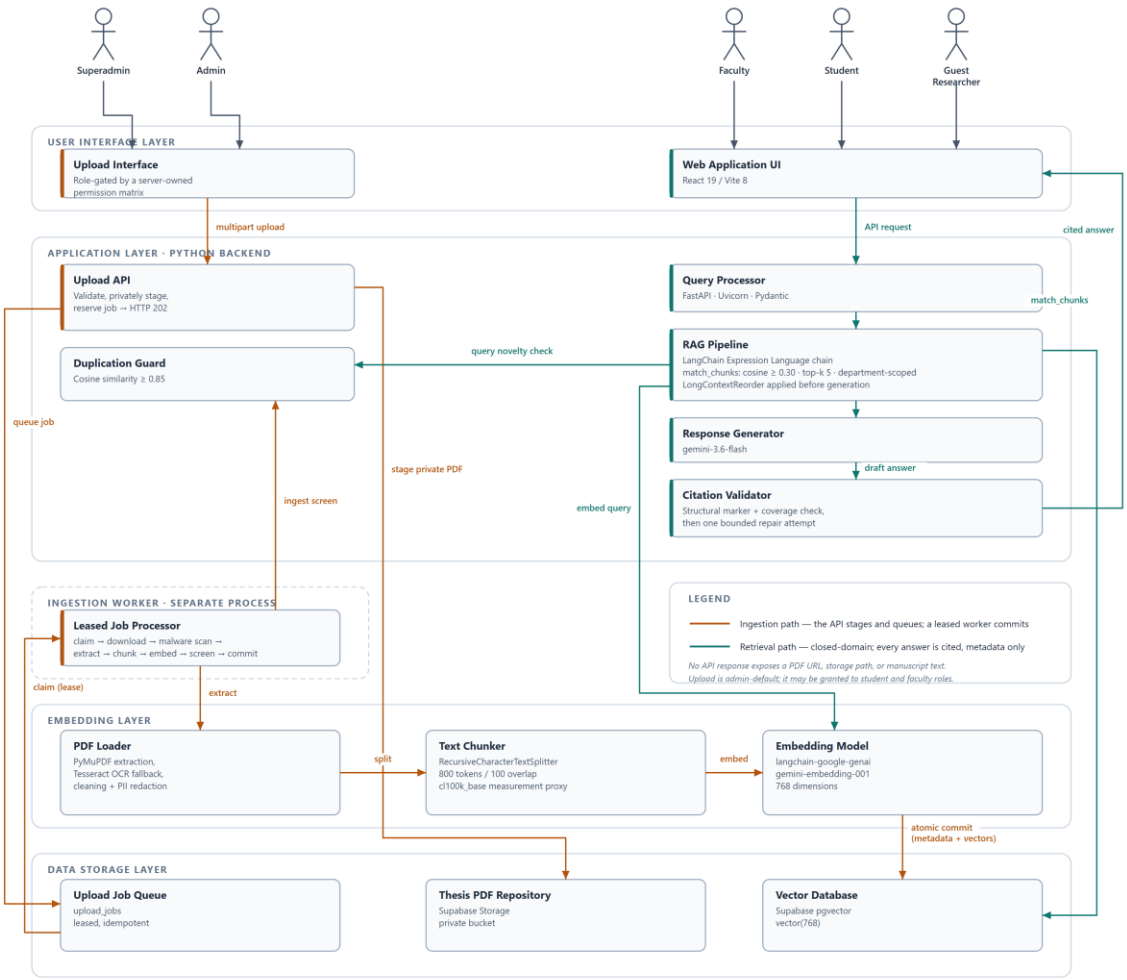
When an administrator uploads a document, the file is first received by the Document Processing System within the Application Layer to handle the multipart upload stream. The data is then routed to the Embedding Layer. Here, a PDF Loader utilizing PyMuPDF will extract the raw text, which will subsequently be divided into manageable segments by a Text Chunker using LangChain's RecursiveCharacterTextSplitter. These chunks will be processed by an Embedding Model, powered by Gemini Embedding, to generate high-dimensional vector representations. These vectors, along with the original PDF files, will be archived securely within the Data Storage Layer, utilizing Supabase pgvector as the designated Vector Database and Supabase Storage as the localized file repository.

During the retrieval phase, when a user submits a question through the UI, the query will be received by the Application Layer. The Query Processor, built on FastAPI, will apply Natural Language Processing (NLP) to interpret the user's intent. This processed query will be sent to the RAG Pipeline, utilizing a LangChain Expression Language prompt-to-model chain, which will search the Vector Database for the most semantically relevant thesis chunks, applying a minimum cosine similarity of 0.30 and returning at most five (top-k = 5). Finally, these retrieved vectors will be fed into the Response Generator via the Gemini 3.6 Flash model, which will synthesize the extracted local data into a coherent, citation-backed answer that will be displayed back to the user. Concurrently, within this Application Layer, the RAG Pipeline calculates the mathematical distance of the retrieved vectors to enforce the 85% duplication

threshold, dynamically flagging the query if the contextual similarity exceeds the allowed redundancy limit.

Figure 8

System Architecture of the Centralized AI-Powered Thesis Library



References

Arivazhagan, M. G., Liu, L., Qi, P., Chen, X., Wang, W. Y., & Huang, Z. (2023). Hybrid hierarchical retrieval for open-domain question answering. *Findings of the Association for Computational Linguistics: ACL 2023*, 10680–10689. https://doi.org/10.18653/v1/2023.findings-acl.679

Armilla, R., & Abangan, M. L. (2025). The difficulties and strategies in digitizing library resources. *Psychology and Education: A Multidisciplinary Journal, 31*(9), 933–962. https://doi.org/10.5281/zenodo.14828861

Aytar, A. Y., Kaya, K., & Kılıç, K. (2025). A retrieval-augmented generation framework for academic literature navigation in data science. *Natural Language Processing Journal, 10*, 100179. https://doi.org/10.1016/j.nlp.2025.100179

Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., & Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN 2024)*. https://doi.org/10.1145/3644815.3644945

Besrour, I., He, J., Schreieder, T., & Färber, M. (2025). SQuAI: Scientific question-answering with multi-agent retrieval-augmented generation. *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM 2025)*. https://doi.org/10.1145/3746252.3761471

Bevara, R. V. K., et al. (2025). Prospects of retrieval augmented generation (RAG) for academic library search and retrieval. *Information Technology and Libraries, 44*(2). https://doi.org/10.5860/ital.v44i2.17361

Bodea, A.-E., Meisenbacher, S., Klymenko, A., & Matthes, F. (2026). SoK: Privacy risks and mitigations in retrieval-augmented generation systems. *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML 2026)*. https://arxiv.org/abs/2601.03979

Brown, A., Roman, M., & Devereux, B. (2025). A systematic literature review of retrieval-augmented generation: Techniques, metrics, and challenges. *Big Data and Cognitive Computing, 9*(12), 320. https://doi.org/10.3390/bdcc9120320

Custer, S., & Sethi, T. (2017). *Avoiding data graveyards: Insights from data producers & users in three countries*. AidData, College of William & Mary. https://www.aiddata.org/publications/avoiding-data-graveyards-insights-from-data-producers-users-in-three-countries

Doliente, C. J. O., et al. (2023). Exploring students' utilization of Online Public Access Catalog in the learning commons. *International Journal of Science and Advanced Information Technology, 12*(4). https://doi.org/10.30534/ijsait/2023/011242023

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 150–165. https://arxiv.org/abs/2309.15217

Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 6465–6488. https://arxiv.org/abs/2305.14627

Gao, Y., Xiong, Y., Dibia, V., et al. (2024). Retrieval-augmented generation for large language models: A survey. *arXiv preprint*, arXiv:2312.10997. https://doi.org/10.48550/arXiv.2312.10997

Gokdemir, O., et al. (2025). HiPerRAG: High-performance retrieval augmented generation for scientific insights. *Proceedings of the 2025 IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. https://doi.org/10.1109/ccgrid64434.2025.00035

Google. (2024). *Gemini API terms of service*. Google for Developers. https://ai.google.dev/gemini-api/terms

Khaliq, A., Abbasi, S. B., Ilyas, A., Shaikh, S. M., & Ali, S. A. (2024). Optimizing academic queries with retrieval-augmented large language models. *Migration Letters, 21*(S10), 1274–1283. https://doi.org/10.59670/ml.v21is10.11858

Kovanen, S. (2025). *Hallucination mitigation for faithful retrieval-augmented generation—Investigating layer-contrastive and attention-based decoding methods* [Master's thesis, Aalto University]. Aaltodoc. https://aaltodoc.aalto.fi

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kutuzov, A., Lewis, M., Yih, W., Rocktäschel, T., & Riedel, S. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474. https://arxiv.org/abs/2005.11401

Li, Z., Wang, Z., Wang, W., Hung, K., Xie, H., & Wang, F. L. (2025). Retrieval-augmented generation for educational application: A systematic survey. *Computers & Education: Artificial Intelligence, 8*, 100417. https://doi.org/10.1016/j.caeai.2025.100417

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157–173. https://doi.org/10.1162/tacl_a_00638

Mezhlumyan, A. (2024). *Building a retrieval-augmented generation (RAG) system for academic papers* [Capstone project, American University of Armenia]. AUA Repository. https://aua.am

Open Web Application Security Project (OWASP). (2025). *OWASP Top 10 for Large Language Model applications*. OWASP Foundation. https://owasp.org/www-project-top-10-for-large-language-model-applications/

Oreški, D., & Vlahek, D. (2024). Retrieval augmented generation in large language models: Development of AI chatbot for student support. *Proceedings of the 15th International Conference on e-Learning (eLearning 2024)*. CEUR Workshop Proceedings, Vol. 3685. https://ceur-ws.org/Vol-3685/

Ru, D., Qiu, L., Hu, X., Zhang, T., Shi, P., Chang, S., Jiayang, C., Wang, C., Sun, S., Li, H., Zhang, Z., Wang, B., Jiang, J., He, T., Wang, Z., Liu, P., Zhang, Y., & Zhang, Z. (2024). RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. *arXiv preprint*, arXiv:2408.08067. https://arxiv.org/abs/2408.08067

Sambrano, L. C., Reyes, J. R. B., Fabro, R. B. B., Colar, C. K. F., & Medrano, V. B. (2025). The influence of artificial intelligence (AI) on library patronage among college students. *International Journal of Research and Innovation in Social Science, 9*(7). https://doi.org/10.47772/IJRISS.2025.90700010

Santra, P. P., Kashyap, R., Mahata, A., & Sk, M. A. (2025). Leveraging retrieval-augmented generation in local library systems: The BiblioGPT prototype. *14th International CALIBER 2025*. https://ir.inflibnet.ac.in/handle/1944/2521

Supabase. (2026). *pgvector: Embeddings and vector similarity search*. Supabase Technical Documentation. https://supabase.com/docs/guides/database/extensions/pgvector

Supreme Court of the Philippines. (2024). *Chief Justice Gesmundo: Judiciary E-Library to use AI technology to improve legal research*. Public Information Office. https://sc.judiciary.gov.ph/chief-justice-gesmundo-judiciary-e-library-to-use-ai-technology-to-improve-legal-research/

Vercruysse, A., Rojas Melendez, J. A., & Colpaert, P. (2024). Linking application and semantic data with RDF Lens. *Proceedings of the 20th International Conference on Semantic*

Systems (SEMANTiCS 2024). CEUR Workshop Proceedings, Vol. 3759. https://ceur-ws.org/Vol-3759/paper13.pdf

Wong, K., et al. (2025). Retrieval-augmented systems can be dangerous medical communicators. *arXiv preprint*, arXiv:2502.14898. https://arxiv.org/abs/2502.14898

Xuan, W., & Shaw, C. (2025). Reimagining library services in the age of AI: A case study from a Canadian academic library. *Proceedings of the IATUL Conferences*, Paper 4. https://doi.org/10.5703/1288284318042